JosephsonCircuitsOptimizer.jl

User Manual

Design, optimization and nonlinear analysis of Josephson circuits

Repository: `SQE-INRiM/JosephsonCircuitsOptimizer` [1]

Project format: `.jco`

Current package series: 0.3.0

Documentation date: September 2026

This document is the user-facing reference for JCO. It explains what JCO is, how a project is stored, how to use the desktop environment, and the syntax required to define and run an experiment. Internal GUI architecture and contributor/developer documentation are intentionally kept separate.

Contents

1 Introduction

JosephsonCircuitsOptimizer.jl (JCO) [1, 2] is a Julia framework for the automated exploration and optimization of superconducting Josephson circuits. It is built on JosephsonCircuits.jl [3], which performs the circuit simulations, while JCO organizes them into a structured workflow for parameter exploration, optimization and nonlinear analysis.

JCO combines three levels of analysis:

1. a computationally efficient linear design-space exploration;
2. a surrogate-based optimization that refines the most promising device parameters;
3. a nonlinear harmonic-balance analysis of the selected design over its physical working points.

This hierarchy allows a broad parameter space to be explored with relatively inexpensive calculations before concentrating the more computationally demanding nonlinear analysis on the most promising designs. The user defines the quantities used to evaluate the circuit, while JCO manages the simulation and optimization workflow.

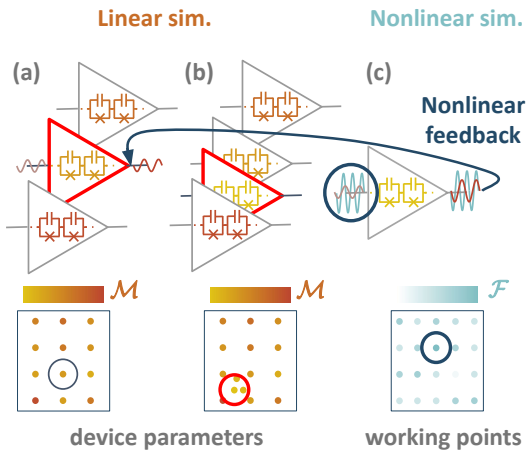

Figure 1: Overview of the JCO workflow. Device parameters are first explored and refined in the linear regime. The selected design is then evaluated under nonlinear operating conditions, with the possibility of feeding nonlinear information back into a subsequent optimization cycle. Adapted from Ref. [2].

1.1 Why JCO?

A complete nonlinear harmonic-balance exploration of every possible device design can be computationally expensive. JCO therefore uses faster linear simulations to evaluate quantities such as impedance matching and phase matching and to build the metric $\mathcal{M}$, which guides the search toward designs expected to provide better nonlinear performance. The most promising device is then evaluated directly in the nonlinear regime through $\mathcal{F}$.

2 Starting JCO

The desktop GUI is launched directly from the repository. Node.js and Electron are prepared locally by the launcher; the user does not need to install Node or npm globally. Julia is intentionally **not bundled** and is required only for operations that execute JCO, such as circuit preview, runtime setup and simulations.

2.1 Windows

From the repository root, double-click:

`START_JCO_GUI.bat`

The launcher prepares the local GUI runtime on first use, builds the interface and opens the Electron application.

2.2 Linux

From the repository root:

```
chmod +x START_JCO_GUI.sh
./START_JCO_GUI.sh
```

The Linux launcher currently supports x86-64 and ARM64/aarch64 systems.

Linux support

Linux support is preliminary and has been less extensively validated than Windows. The launcher is provided so the same JCO project can be used on Linux, but distribution-specific Electron libraries or other platform-specific issues may still need manual resolution.

2.3 Julia runtime and direct execution

For the desktop GUI, Julia remains external. Set the Julia executable and the desired number of threads from the GUI settings; runtime setup can then instantiate and precompile the JCO environment.

JCO can also be run directly from Julia. From the repository root:

```
using Pkg
Pkg.activate(".")

import JosephsonCircuitsOptimizer as JCO
JCO.run()
```

A different workspace can be selected with:

```
JCO.run(workspace=raw"C:\...\my_experiment_01")
```

When Julia is started manually, the thread count is selected at launch, for example:

```
julia --threads 12
```

and can be checked inside Julia with `Threads.nthreads()`.

3 The JCO project file

3.1 The `.jco` file

A JCO experiment is normally stored as a single file with extension `.jco`. The file is a versioned ZIP container rather than a new monolithic numerical format. Internally, it preserves the same editable JCO workspace used by the Julia package.

This choice has two advantages: the user can move or archive one project file, while JCO can still work with ordinary Julia, JSON, text and HDF5 files inside the project.

A project contains two main classes of information:

Area	Contents
Project manifest	Project identity, format version and timestamps.
User inputs	Circuit definition, device-parameter space, sources, solver/optimizer settings, metric/performance functions, optional helper code and auxiliary text data.
Outputs	One directory for every retained simulation run. Each run contains numerical summaries, saved arrays, optimal parameters and run metadata.

Table 1: High-level contents of a JCO project.

Opening a `.jco` file extracts it into a private working directory used by the desktop application. Opening the file does **not** execute the Julia code stored in the project. The archive is repacked when the project is saved and after completed scientific runs.

3.2 Input files

The standard experiment definition is built from six required files:

- `device_parameters_space.json`;
- `drive_physical_quantities.json`;
- `simulation_config.json`;
- `optimizer_config.json`;
- `user_circuit.jl`;
- `user_cost_and_performance.jl`.

Optional Julia helper files, parametric-source functions and auxiliary calibration/text files can be stored with them. The GUI edits these files through structured controls where possible while preserving the underlying JCO representation.

3.3 Runs and results

A project can contain multiple run directories. A new run does not overwrite the previous one, so the user can change the setup, launch another calculation and later return to an earlier result.

For each run, JCO retains compact numerical summaries and metadata. The principal files used by the GUI are shown in Table 2.

A stable `point_id` links each row in a summary table to the detailed arrays saved for that same simulation point. This allows Results to open the correct trace without comparing floating-point parameter values. Scalar quantities returned by `user_cost` and `user_performance` are stored in the compact summary tables. Larger frequency-resolved arrays, such as a selected $S_{21}(f)$ trace, can be retained explicitly with `save_data`; these arrays are linked to the corresponding summary-table point through `point_id`. The syntax is described in Sec. 5.10.

File	Purpose
`df_uniform_analysis.h5`	Linear design-space summary: device coordinates, stable `point_id`, objective and additional scalar metrics.
`df_optimization_analysis.h5`	Optimizer evaluations: evaluation number, sampled device parameters and objective value.
`df_nonlinear_analysis.h5`	Harmonic-balance summary: source working-point coordinates, convergence, performance and additional scalar quantities.
`saved_data/linear.h5`	Detailed arrays that the user explicitly chose to retain for Linear points.
`saved_data/nonlinear.h5`	Detailed arrays that the user explicitly chose to retain for nonlinear points.
`optimal_device_parameters.json`	Device parameters selected by the optimization.
`optimal_physical_quantities.json`	Selected nonlinear operating point when available.

Table 2: Principal run outputs used by the GUI. Long filenames are shown with breakable paths to avoid table overlap.

3.4 Legacy workspaces and export

An existing JCO workspace can be imported into the GUI and then saved as a `.jco` project. The editable workspace can also be exported as a ZIP archive. Numerical data can be exported separately from Results, for example to CSV, without changing the project archive.

4 The JCO environment

The desktop interface is intentionally organized around three areas: **Setup**, **Run** and **Results**. The same experiment can therefore be configured, executed and inspected without manually navigating the underlying workspace.

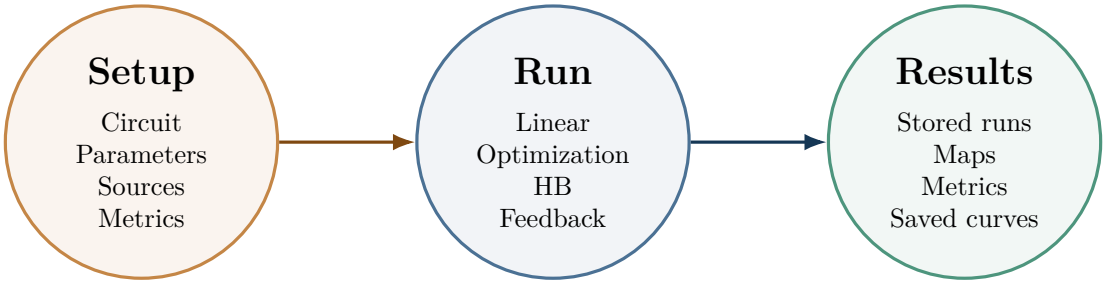

Figure 2: The three user-facing areas of the JCO desktop environment. Setup defines the experiment, Run executes the selected pipeline, and Results inspects the numerical data stored in the project.

4.1 Setup

Setup defines the experiment before computationally expensive work begins (Fig. 3). It is divided into three steps.

4.1.1 Model & inputs

This step contains the circuit, device parameters and sources.

- **Circuit**: edit `user_circuit.jl` and request a lumped-element preview. The preview is generated explicitly; simply opening a project never executes Julia.

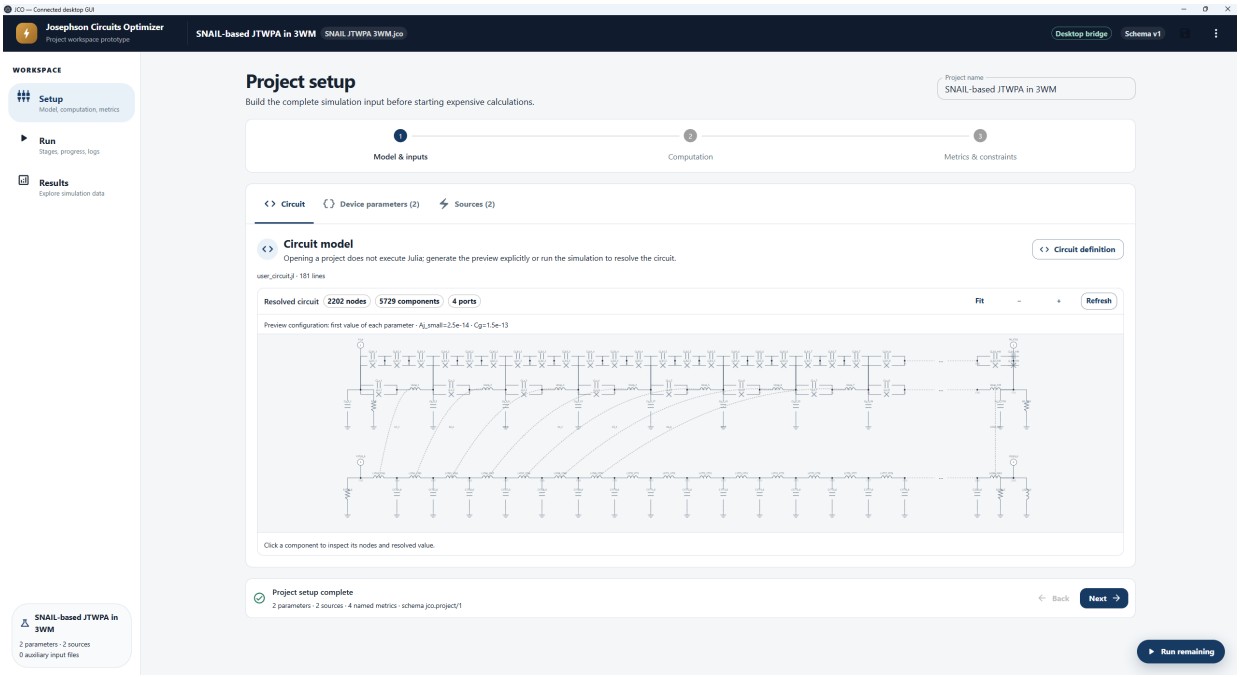

Figure 3: Screenshot of the GUI Setup section

- **Device parameters**: define every parameter as Fixed, Range or List. The GUI reports the total number of device configurations entering the Linear sweep.
- **Sources**: define ports, AC/DC behavior, frequency and Linear/nonlinear amplitudes. Amplitudes can be displayed in A or dBm and can also be generated by a Julia function of the current device parameters.

The circuit preview is useful as a structural check before a long run: it resolves the current model and displays the lumped-element network corresponding to the selected parameter set.

4.1.2 Computation

This step collects settings that control the simulation rather than the physical model:

- Linear frequency interval and resolution;
- optimizer iterations, samples, surrogate, strategy and sampler;
- Linear and nonlinear harmonic truncation;
- common harmonic-balance solver tolerances and iteration limits;
- three-wave/four-wave mixing options;
- number of nonlinear-feedback cycles.

When all device parameters are fixed, the project is a single-point experiment and the optimization stage is skipped automatically.

4.1.3 Metrics & constraints

The GUI reads the named values returned by `user_cost_and_performance.jl`. The first value returned by `user_cost` is the Linear objective and is minimized; the first value returned by `user_performance` is the nonlinear objective and is maximized. Additional named values are retained as analysis metrics.

Constraints and masks remain part of the user cost definition. If the measured quantity behind a constraint is also returned as a named value, it becomes available in the stored dataset and Results page.

4.2 Run

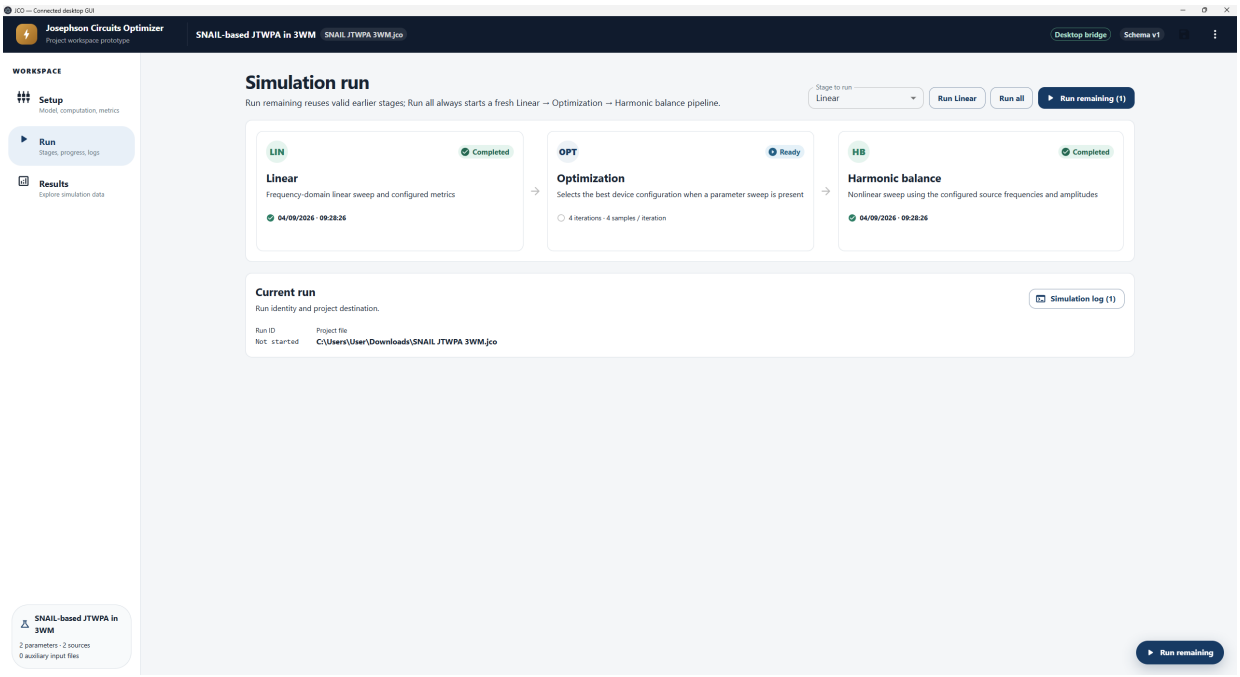

Figure 4: Screenshot of the GUI Run section.

The Run page (Fig. 4) shows the three computational stages:

$$\textbf{Linear} \longrightarrow \textbf{Optimization} \longrightarrow \textbf{Harmonic balance}$$

For each stage the interface reports status, progress, completion time and duration. During a run, the full Julia log remains accessible without occupying the main screen.

Three execution controls are intentionally different:

- **Run all**: starts a fresh complete pipeline and does not reuse an earlier Linear or Optimization result.
- **Run remaining**: continues from earlier results only when those stages are still valid for the current setup.
- **Run stage**: executes the selected stage only, using the stage-specific JCO entry point and the required existing data where applicable.

Changing an input that affects a completed stage marks dependent stages as stale. This prevents a visually "completed" stage from being silently reused after the experiment definition has changed.

A real scientific run requires the project to have been saved at least once. The displayed run ID is the same identifier used by the corresponding output directory. If the user requests cancellation, JCO first asks Julia to stop cleanly and then terminates the process tree if the solver does not respond within the grace period.

4.3 Results

Results are organized by stage and by stored run (Fig. 5). Earlier completed runs remain accessible even while a new simulation is running.

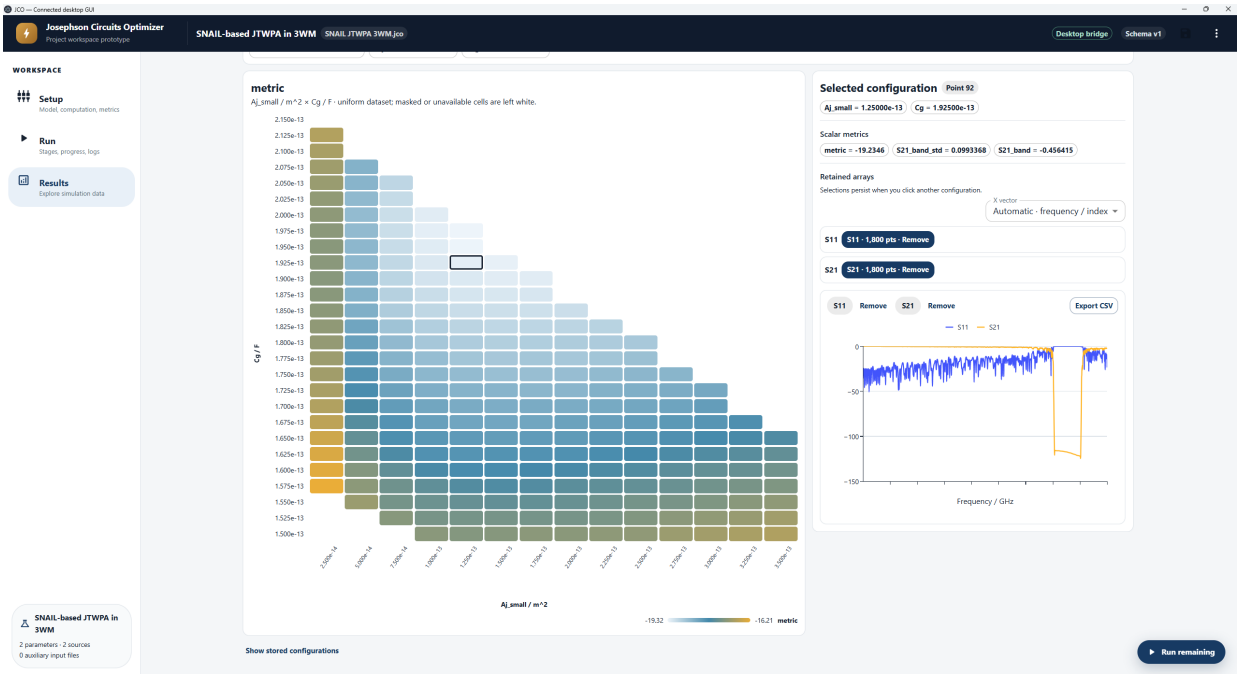

Figure 5: Screenshot of the GUI Results section.

4.3.1 Linear results

The Linear view works on the stored uniform design-space table. Depending on dimensionality, the user can inspect:

- 1D/2D design landscapes for a chosen metric;
- exact slices of higher-dimensional parameter spaces by fixing the remaining coordinates;
- the selected point's device coordinates, resolved circuit parameters and scalar metrics;
- user-saved arrays, plotted on the stored frequency axis when compatible;
- parameter/metric correlation and JCO-style one-dimensional density analysis.

Higher-dimensional slices are exact: hidden coordinates are fixed to stored values rather than averaged or interpolated.

4.3.2 Optimization results

The Optimization view is deliberately limited to optimizer diagnostics: the best sampled objective versus evaluation number and the raw history of each optimized parameter. Design-space correlation belongs to the Linear uniform dataset rather than to the non-uniform optimizer samples.

4.3.3 Harmonic-balance results

The Harmonic Balance view explores the stored nonlinear working-point space. Source frequency and amplitude can be selected as axes, other source coordinates can be fixed explicitly, and results can be filtered by solver convergence. The selected point exposes nonlinear metrics, device parameters and saved arrays. Compatible saved arrays can be plotted using a frequency axis or another selected saved array as the x-axis.

4.3.4 Run history

The setup snapshot attached to a run shows the input configuration that produced that result, not the setup currently being edited. Stored runs can be managed and deleted from the project; the

currently active simulation is protected from deletion.

5 Building and running an experiment

The recommended way to prepare an experiment is through the GUI. The underlying Julia and JSON files are nevertheless kept visible because they are the scientific definition of the experiment and make JCO projects transparent, scriptable and usable without the desktop interface.

In the following sections, each item is therefore described in two layers: **what the user edits in Setup** and **the file/syntax that the GUI reads or writes**.

The GUI is the recommended way to compile these inputs. The following sections therefore start from the corresponding GUI control and then show the Julia or JSON representation written inside the project. This makes the GUI the main user interface without hiding the experiment files on which JCO is based.

5.1 Circuit definition

In the GUI. Open **Setup** → **Model & inputs** → **Circuit**. The **Circuit definition** button opens `user_circuit.jl`. After the code is valid and Julia is configured, **Generate preview** evaluates the current circuit and displays its lumped-element scheme. This is a useful check of ports, nodes, repeated elements and parameter-dependent component values before starting a long sweep.

Underlying Julia syntax. The circuit file must define:

```
function create_user_circuit(device_params_set)
    # build circuit and circuitdefs
    return circuit, circuitdefs
end
```

A minimal one-port example is:

```
function create_user_circuit(device_params_set)

    @variables R Cc Lj Cj

    circuit = [
        ("P1",  "1", "0", 1),
        ("R1",  "1", "0", R),
        ("C1",  "1", "2", Cc),
        ("Lj1", "2", "0", Lj),
        ("C2",  "2", "0", Cj)
    ]

    circuitdefs = Dict(
        Lj => device_params_set[:Lj],
        Cc => device_params_set[:Cc],
        Cj => device_params_set[:Cj],
        R  => 50.0
    )

    return circuit, circuitdefs
end
```

Each circuit tuple gives the component name, its two nodes and the component definition expected by JosephsonCircuits. Port elements use names beginning with `P`; the GUI uses the resolved circuit to count ports and construct the preview.

Circuit code and the Device parameters panel

The connection between circuit code and the Device parameters panel is automatic. The GUI scans `user_circuit.jl` for keys accessed through the circuit parameter dictionary. For example, writing `device_params_set[:Cj]` makes `Cj` appear automatically in **Setup** → **Device parameters**. If the corresponding value is not yet present in `device_parameters_space.json`, the GUI marks it as *Needs value*. Entering its Fixed/Range/List value in the GUI writes the corresponding JSON entry. Existing JSON values are loaded back into the same controls when the project is reopened.

Parameters may also be modified or derived during circuit creation when that is part of the physical model. JCO stores the resolved post-circuit scalar values for Linear points, so Results can distinguish the sweep coordinates from values calculated inside `create_user_circuit`.

5.2 Device parameters: Fixed, Range and List

In the GUI. The **Device parameters** panel is normally the easiest place to define the design space. Every circuit parameter detected from `user_circuit.jl` can be assigned one of three modes:

- **Fixed**: one value, used in every design;
- **Range**: start, stop and step;
- **List**: an explicit set of values.

The GUI immediately reports how many circuit configurations are generated by the Cartesian product of all non-fixed parameters. Units entered in the GUI are display metadata; the numerical values written to the JCO input file remain the values used by the Julia model.

Underlying JSON. A fixed value is stored as a one-element list:

```
{
  "alpha": [0.16]
}
```

A regular range uses `start`, `step` and `stop`:

```
{
  "Lj": {"start": 1.0e-9, "step": 0.1e-9, "stop": 1.2e-9}
}
```

An explicit list stores all requested values:

```
{
  "Cg": [100e-15, 150e-15, 220e-15]
}
```

Several parameters can be combined in the same file. JCO forms their Cartesian product for the initial Linear exploration. The name in the JSON file is the same name used in `device_params_set[:name]` in the circuit.

5.3 Sources and frequency range

In the GUI. Use **Setup** → **Model & inputs** → **Sources** to add sources, select their ports, mark DC sources, and define frequency plus Linear/nonlinear amplitudes as Fixed, Range or List. Amplitudes can be displayed in A or dBm; the GUI performs the conversion before saving the JCO input. Parametric source functions can also be selected from the same panel.

Underlying JSON. The drive file contains the small-signal frequency range and one indexed group of keys for each source. A minimal single-source configuration is:

```
{
  "frequency_range": {
    "start": 4.5e9,
    "step": 1.0e6,
    "stop": 5.0e9
  },

  "source_1_on_port": 1,
  "source_1_frequency": 4.75e9,
  "source_1_linear_amplitude": 5.65e-9,
  "source_1_non_linear_amplitude": [0.565e-6, 1.695e-6, 5.65e-6]
}
```

Source frequencies and amplitudes accept Fixed, Range or List semantics through the GUI. A DC source has zero frequency. For nonlinear sweeps, multiple source frequencies and amplitudes generate a working-point space that can later be sliced in Results.

The GUI can display amplitudes in amperes or dBm. dBm values are converted to the current amplitude stored by JCO using the selected source impedance Z_0:

$$I = \sqrt{\frac{P}{2Z_0}}, \qquad P = 10^{-3}10^{P_{\mathrm{dBm}}/10}. \tag{1}$$

The GUI default is $Z_0 = 50\,\Omega$. The value `-Inf` dBm represents an off-state, i.e. zero current.

5.4 Parametric sources

In the GUI. A Linear or nonlinear source amplitude can be set to **Parametric**. The **Parametric functions** editor contains `user_parametric_sources.jl`; the GUI lists functions whose first argument is `device_params_set` (or `device_parameters_set`) and uses the selected function name as the source amplitude. The function is evaluated for every device configuration and must return a source current in amperes.

Calibration data can be attached from the same Sources panel with **Add .txt**. The file is stored inside `user_inputs/` in the `.jco` project and is therefore available when the parametric function is evaluated.

Underlying Julia syntax. A useful case is a source amplitude obtained from a calibration curve. For example:

```
using ..Config

path = joinpath(config.user_inputs_dir, "flux_curve.txt")
lines = readlines(path)

alpha_flux_map = map(
    line -> round(parse(Float64, split(line, ",")[1]), digits=2),
    lines
)
flux_map = map(
    line -> round(parse(Float64, split(line, ",")[2]), digits=2),
    lines
)

global interp_alpha_flux = linear_interpolation(
    alpha_flux_map,
    flux_map,
```

```
    extrapolation_bc=Flat()
)

function find_flux_from_alpha(alpha)
    return interp_alpha_flux(alpha)
end

function calculate_source_1_amplitude(device_params_set::Dict)
    phidc = find_flux_from_alpha(device_params_set[:alphaSNAIL])
    source_1_amplitude = phidc * (2 * 280 * 1e-6)
    return source_1_amplitude
end
```

Only `calculate_source_1_amplitude` is offered as a source binding because it receives the device-parameter dictionary as its first argument; helper functions such as `find_flux_from_alpha` remain available in the file but are not listed as source functions.

When the source is saved, the underlying drive configuration stores the function name, for example:

```
"source_1_non_linear_amplitude": "calculate_source_1_amplitude"
```

This pattern is useful whenever the physical drive must change together with a device parameter rather than remaining fixed over the design sweep.

5.5 Simulation settings

In the GUI. Most simulation controls are available under **Setup** → **Computation**: frequency interval, Linear/nonlinear harmonics, solver iterations/tolerances, node sorting, three-wave/four-wave mixing and the number of nonlinear-correction cycles. These fields are written to `simulation_config.json` when the project is saved.

Underlying JSON. A representative simulation configuration is:

```
{
  "linear_strong_tone_harmonics": 1,
  "linear_modulation_harmonics": 1,
  "nonlinear_strong_tone_harmonics": 16,
  "nonlinear_modulation_harmonics": 8,
  "max_simulator_iterations": 300,
  "ftol": 1e-8,
  "switchofflinesearchtol": 1e-5,
  "alphamin": 1e-4,
  "maxintermodorder": "Inf",
  "nbatches": 0,
  "sorting": "number",
  "threewavemixing": false,
  "fourwavemixing": true,
  "n_iterations_nonlinear_correction": 0
}
```

The harmonic settings are stage-specific, while the main solver tolerances are shared. With `nbatches = 0`, JCO allows the solver to use the available Julia threads.

5.6 Optimizer settings

In the GUI. The Optimization card in **Setup** → **Computation** exposes the number of iterations, samples per iteration, random seed, surrogate model, acquisition/search strategy and sampler. When all device parameters are fixed, the GUI reports that Optimization will be skipped.

Underlying JSON. The optimizer configuration controls the surrogate search after the initial Linear dataset:

```
{
  "max_optimizer_iterations": 5,
  "new_samples_per_optimizer_iteration": 5,
  "surrogate_model": "Kriging",
  "optimizer_strategy": "SRBF",
  "sampling_strategy": "RandomSample",
  "random_seed": 1234
}
```

Currently supported choices are:

- surrogate: `Kriging`, `RadialBasis`, `SecondOrderPolynomial`;
- strategy: `SRBF`, `EI`, `LCBS`;
- sampler: `RandomSample`, `SobolSample`, `LatinHypercubeSample`.

The optimizer uses the objective defined by `user_cost`. Additional analysis columns and `point_id` are not optimization variables.

5.7 Linear metric $\mathcal{M}$

In the GUI. Open **Setup → Metrics & constraints → Cost & performance**. The editor contains `user_cost_and_performance.jl`. The GUI reads the names returned by `user_cost`; the first returned value is shown as the objective to minimize and later names appear as analysis metrics.

Underlying Julia syntax. The Linear objective is defined by:

```
function user_cost(sol, device_params_set, nonlinear_correction)
    # extract the desired response from sol
    # build the metric M

    return (
        metric = metric,
        another_quantity = another_quantity,
    )
end
```

For example, a simple metric based on the average input reflection can be written as:

```
function user_cost(sol, device_params_set, nonlinear_correction)
    S11 = sol.linearized.S((0,), 1, (0,), 1, :)
    impedance_match = mean(20 .* log10.(abs.(S11)))

    metric = impedance_match + nonlinear_correction

    return (
        metric = metric,
        impedance_match = impedance_match,
    )
end
```

In this example the optimizer minimizes the average S_{11} in dB, so more negative values correspond to better input matching. The same structure can be used for a metric combining several linear quantities.

`sol` is the JosephsonCircuits solution for the current device configuration. The first returned value is the optimization objective and is **minimized**. Additional named values are saved as scalar analysis metrics.

The third argument, `nonlinear_correction`, is zero in the initial Linear pass. When nonlinear feedback is enabled, JCO replaces it with the correction produced by the nonlinear stage before repeating the Linear optimization. How that correction modifies $\mathcal{M}$ is deliberately under user control.

Constraints can be applied directly inside `user_cost` with JCO's `mask` helper. A typical pattern is:

```
input_mask = (
    second_harm = second_harm,
    impedance_match = impedance_match
)

conditions_mask = x ->
    x.second_harm < 1 &&
    x.impedance_match < -10

if mask(input_mask, conditions_mask)
    return 1e8
end
```

The condition function describes the region that is accepted. If the conditions are satisfied, `mask` returns `false` and the normal metric calculation continues. If they are not satisfied, the point is counted as excluded and `mask` returns `true`; the cost function can then return a large penalty such as `1e8`. In this example, only points satisfying both the second-harmonic and impedance-matching requirements are allowed to compete in the optimization.

When a constrained quantity is also useful for analysis, it can additionally be returned as a named value from `user_cost` so that it appears in the Linear result table.

5.8 Nonlinear performance $\mathcal{F}$

In the GUI. The same **Cost & performance** editor contains `user_performance`. The GUI recognizes its returned names: the first is the nonlinear performance to maximize, while later values are displayed as additional HB metrics.

Underlying Julia syntax. The nonlinear working-point objective is defined by:

```
function user_performance(sol, device_params_set, source_amps, source_freqs)
    # evaluate nonlinear response

    return (
        performance = performance,
        another_quantity = another_quantity,
    )
end
```

The first returned value is the performance $\mathcal{F}$ and is **maximized** when JCO identifies the best nonlinear working point. Additional named values become nonlinear analysis metrics.

The source arrays correspond to the exact source amplitudes and frequencies used at the current HB point. They can therefore be used directly when the desired performance depends on the operating condition as well as on the circuit response.

5.9 Nonlinear feedback

JCO expects a nonlinear-correction hook:

```
function user_nonlinear_correction(S_linear, sol, device_params_set)
    delta_M = 0.0   # user-defined correction
    return delta_M
end
```

When feedback cycles are enabled, JCO evaluates this term for the selected device and reference nonlinear amplitudes, passes it back as `nonlinear_correction` to `user_cost`, repeats the Linear sweep/optimization around the current optimum, and then evaluates the new nonlinear working points. Returning `0.0` leaves $\mathcal{M}$ unchanged.

Reference amplitudes for the correction can be included in the source configuration using keys such as:

```
"source_1_non_linear_amplitude_for_delta_correction": 0.565e-6
```

5.10 Saving frequency-resolved or custom arrays

In the GUI. There is no need to predefine every possible frequency-resolved result in the interface. The experiment code chooses which arrays are worth retaining; after the run, those arrays appear automatically for the corresponding selected point in Results.

Underlying Julia syntax. Scalar values returned by `user_cost` and `user_performance` are already stored in the summary HDF5 tables. Larger arrays should be saved explicitly only when they will be useful in Results.

Inside a Linear or nonlinear point, use:

```
save_data(S21)
```

or the explicit macro form:

```
@save_data S21
```

For GUI-loaded metric code, a simple `save_data(S21)` call preserves the variable name so the Results catalogue displays `S21` rather than a generic label. The active stage is selected automatically. Arrays are associated with the current stable `point_id` and written to `saved_data/linear.h5` or `saved_data/nonlinear.h5`.

A common frequency axis is stored when it matches the standard JCO frequency grid. Complex arrays are retained as real and imaginary parts; Results can display magnitude, real part, imaginary part or phase without storing separate plot images.

What should be saved?

Return compact scalar quantities from `user_cost`/`user_performance`. Use `save_data` for the smaller set of arrays that you may want to inspect later. Do not save every solver object or every S-parameter trace by default.

5.11 Optional helper and calibration files

Reusable metric functions can be placed in `user_metric_utils.jl`; source functions can be placed in `user_parametric_sources.jl`. Plain-text calibration files can also be attached from Setup and are stored inside the project input directory. These files travel with the `.jco` archive.

5.12 Practical run sequence

A typical new experiment is run as follows:

1. Create a new project or import an existing JCO workspace.

2. Define the circuit and verify the lumped-element preview.
3. Define Fixed/Range/List device parameters.
4. Define source ports, frequencies and Linear/nonlinear amplitudes.
5. Configure the frequency range, harmonics, solver and optimizer.
6. Define `user_cost`, `user_performance` and the nonlinear-correction hook; save only the detailed arrays that are useful for later analysis.
7. Save the project as a `.jco` file.
8. Configure the Julia executable/threads and perform runtime setup if needed.
9. Use **Run all** for a fresh complete experiment. Use **Run remaining** only when you intentionally want valid previous stages to be reused, or **Run stage** for a specific stage-only calculation.
10. Inspect Linear, Optimization and HB datasets in Results and export selected numerical data when required.

Quick troubleshooting

Symptom	Check
GUI does not start	Delete the local `gui/.jco-runtime` only if the launcher reports an incomplete Node runtime; otherwise read the launcher error before changing the Julia environment.
Julia executable not found	Set the Julia executable in GUI settings to `julia` or to the full executable path.
First simulation fails during package loading	Run the Julia runtime setup and verify internet access/package installation.
A stage is marked stale	An input affecting that stage changed. Re-run it or use Run all for a fresh pipeline.
Optimization is skipped	All device parameters are fixed; the project is a single-point experiment.
No retained trace is shown	Scalar results may exist even when no `save_data` call was made for that point.
HB point missing	Check the convergence filter and solver log; non-converged working points may be absent or filtered.
Linux Electron launch fails	Linux support is preliminary; check distribution-specific Electron/system-library requirements.

Developer documentation

This manual intentionally stops at the experiment/user boundary. Electron architecture, React state management, Julia bridge internals, provenance implementation, test architecture, repository governance and extension patterns should be documented separately as developer documentation rather than mixed into the user guide.

References

[1] JosephsonCircuitsOptimizer.jl source repository, https://github.com/SQE-INRiM/JosephsonCircuitsOptimizer.

[2] E. Palumbo *et al.*, "JosephsonCircuitsOptimizer.jl (JCO)," *IEEE Transactions on Applied Superconductivity*, 2026. DOI: 10.1109/TASC.2026.3679488.

[3] JosephsonCircuits.jl documentation and source repository, https://josephsoncircuits.org/.